Change Impact Analysis & Combinatorial Testing

Slides by Sam Allen

The Testing Boundary





Part 1: Prelude: Change Detection Analysis

Change Detection Analysis

- Change detection analysis seeks to find how changes to individual model constants affect model outputs.
- It does this by changing a single scalar model constant across its range of values while holding all other model constants at their default values.
- Provides a simple form of qualitative analysis (one-constant-at-a-time) but misses interactions among model constants and their joint effect on output changes

Assumptions

- Initial analysis is built on one key assumption.
  - The model constants' effects on model outputs aren't determined by other preconditions being met, such as specific model inputs having certain values
- By discussing with the domain scientists and examining `LakeTemperature` source code, we found this assumption does not hold, meaning the results of change detection do not necessarily include all the model outputs that the model constants affect.

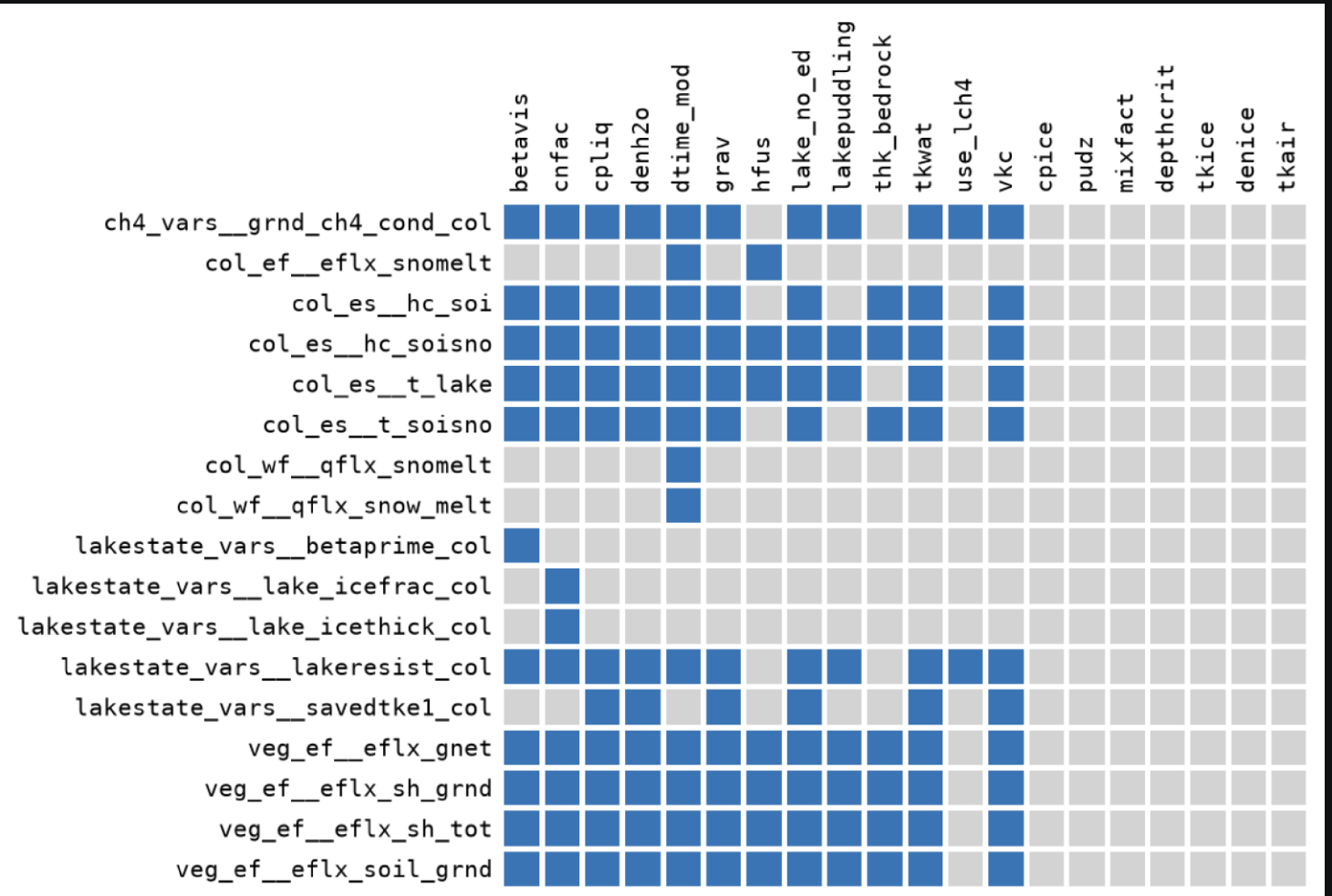
Usefulness (Despite the Limitations)

- This limitation doesn’t discount the results of our change detection analysis.
- It led to the discovery that certain model constants only affect model outputs under certain conditions.
- The results are still valid and useful when the model constants are using the default values provided by the domain scientists.

Rationale

- To determine which model constants affect which model outputs, 11 cases were created for every float model constant and 2 cases were created for every Boolean model constants, where all model constants were held at their default values except the one which was being tested, which was linearly interpolated to get values evenly spread across its valid range.
- This assumed that model constants are independent of each other, and that no preconditions need to be met for a constant to have an effect on a model output.

Preliminary Results



- 58 out of 75 model constants did not change at all.
- We expect some model outputs to stay the same, like bounds variables.
- These results would have to be interpreted by someone with domain knowledge to find the potential issues they expose in `LakeTemperature`

Part 2: Change Impact Analysis

Change Impact Analysis

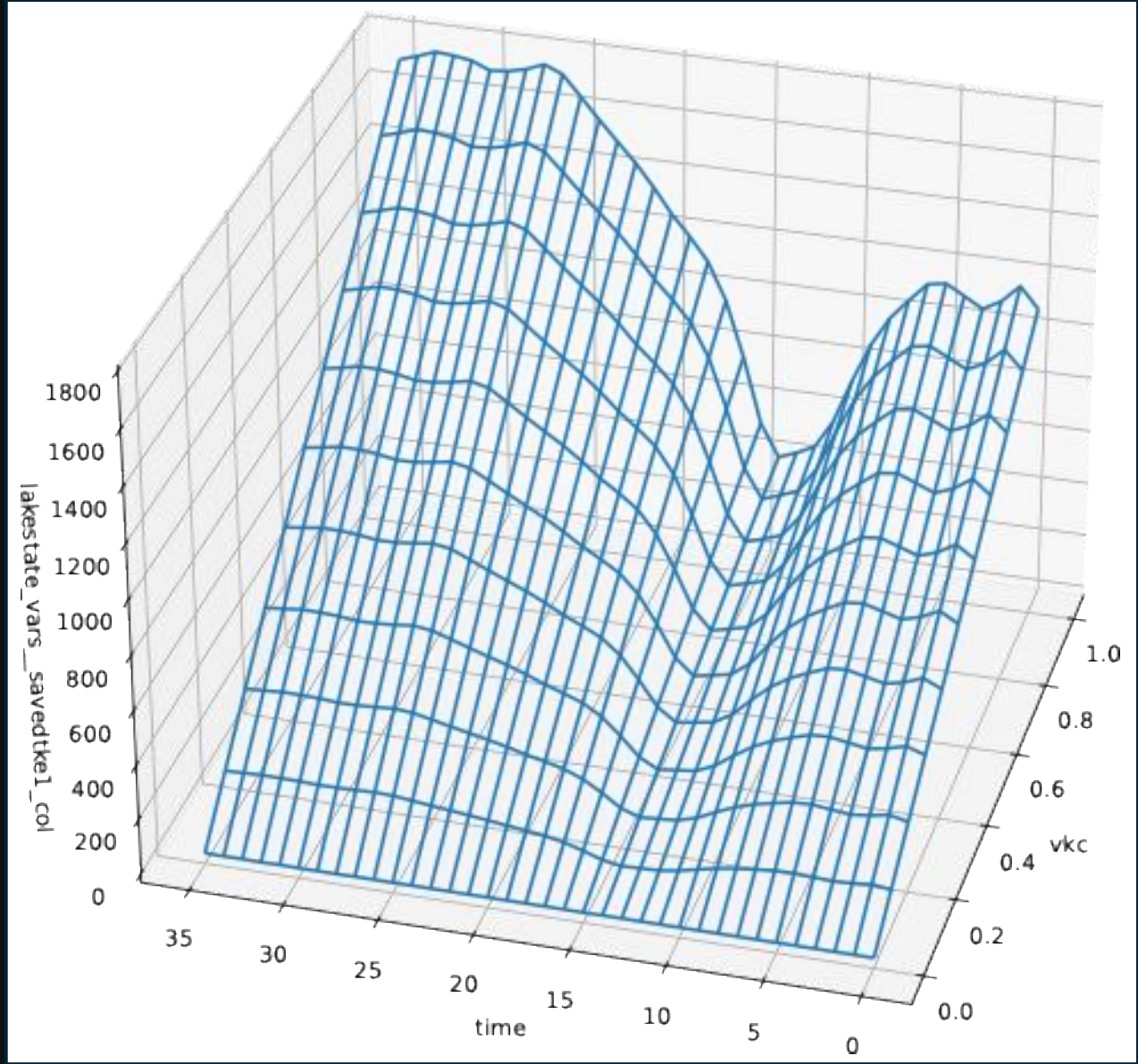
- The goal is to visualize the impact of scalar model constants on model outputs.
- Some challenges are:
  - Most of the 20 model constants we change affect multiple model outputs.
  - The 75 model outputs are multidimensional. The column dimension goes from 0 to 344.
  - Contrary to an initial assumption, some model constants only affect model outputs when certain preconditions are met.
- We created a 3D graph using time as the x-axis, the one model constant we changed as the y-axis, and output data from a single column of a single model output as the z-axis. For model outputs which also have a layer dimension, this will be held constant as well.

Graphing Process

- Output data in the form of a NetCDF file was used.
- From the output data we pulled the values for the model constant *vkc*, which represents the *Von Kármán Constant.*
- Time was interpolated from 0 to 35.
- From the output data we pulled the values for the model output *lakestate_vars__savedtke1_col*–the *eddy conductivity* at the top lake level saved for the next step–at the column numbered 336.
- Using matplotlib, all three dimensions were graphed.

An Example Graph

- This graph shows how the model constant *vkc* affects the model output *lakestate_vars__savedtke1_col* over a period of 35 time steps for the lake column numbered 336.



Part 3: Combinatorial Testing

Why Combinatorial Testing (CT)?

- Ideally, `LakeTemperature` should be validated against all possible combinations of model constant values to ensure that no combination results in erroneous output.
- Exhaustive testing is neither practical nor feasible due to the vast number of all possible combinations.
- Combinatorial testing substantially reduces the number of required test cases while achieving near-exhaustive coverage.
- Assuming most of the bugs are triggered by the interactions of model constants

Background of Combinatorial Testing

- NIST research found most software bugs are caused by the interactions of only a small number of parameters[1].
- By testing all combinations of a certain number of parameters, most software bugs can be found with much fewer test cases.
- NIST's ACTS tool can generate covering arrays from 1-way to 6-way or with mixed coverage[2].

1. *Automated Combinatorial Testing for Software (ACTS)*. NIST Computer Security Division. (2016, July 20). https://csrc.nist.rip/groups/SNS/acts/
2. *User Guide for ACTS*. NIST. (n.d.). https://csrc.nist.gov/csrc/media/Projects/automated-combinatorial-testing-for-software/documents/acts_user_guide_for_basic1.0_and_advanced3.3_versions_nov23.pdf

Application of CT to `LakeTemperature`

- The scalar model constants, which we can control when running `LakeTemperature`, are used as the parameters for the combinatorial test cases.
- 3 scalar model constants are Booleans, 7 are integers, and 15 are floats.
- Booleans can be either 0 or 1.
- For integers, we linearly interpolated over their ranges to get up to 11 values.
- For floats, we linearly interpolated over their ranges to get exactly 11 values.

NetCDF Excerpts for Illustration

```
bool lakepuddling = 0, 1 ;

int nlevsno = 0, 1, 2, 3, 4, 5 ;

int depthcrit = 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35 ;

float thk_bedrock = 2, 2.2, 2.4, 2.6, 2.8, 3, 3.2, 3.4, 3.6, 3.8, 4 ;
```

Test Oracle

- Scientific software verification is challenging due to often lacking a test oracle.
- We implemented checks that validate model outputs, supported by the fault finding analysis plugin.
- Checks include ranges of outputs, signs, enumerated values, finiteness, relationships to other outputs, and certain physical laws and constraints.

Sign Checks Overview

- Check that a certain model output is not negative
- 11 out of 55 checks are of this type.

Example Check

```python
def check_water_snow_equivalent_not_negative(
    test_col_ws_h2osno: npt.NDArray
) -> None:

    assert_not_negative(test_col_ws_h2osno, "col_ws%ws_h2osno")
```

```python
def assert_not_negative(variable: npt.NDArray, name: str) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(variable):
        return CheckStatus.SKIPPED
    variable = NonFiniteValuesHandler.mask_non_finite_values(variable)
    assert np.all(variable >= 0.0), f"Negative values found in {name}"
```

All Sign Checks

- $\forall t \forall c(ch4_vars\%grnd_ch4_cond_col[t, c] \geq 0)$ CH4 Conductance Behavior Check
  - ch4_conductance_check.check_methane_conductance_not_negative
  - Precondition: use_lch4 = 1
- $\forall t \forall c(col_ws\%h2osno[t, c] \geq 0)$ Physical Bounds Check
  - physical_bounds_check.check_water_snow_equivalent_not_negative
- $\forall t \forall c(col_ws\%snow_depth[t, c] \geq 0)$ Physical Bounds Check
  - physical_bounds_check.check_snow_depth_not_negative
- $\forall t \forall j \forall c(col_wf\%qflx_snofrz_lyr[t, j, c] \geq 0)$ Physical Bounds Check
  - physical_bounds_check.check_snow_freeze_rate_not_negative
- $\forall t \forall c(col_wf\%qflx_snomelt[t, c] \geq 0)$ Physical Bounds Check
  - physical_bounds_check.check_snow_melt_flux_not_negative

All Sign Checks (Continued)

All Physical Bounds Checks

- $\forall t \forall c(col_wf\%qflx_snow_melt[t, c] \geq 0)$
  - physical_bounds_check.check_net_snow_melt_not_negative
- $\forall t \forall c(col_ef\%eflx_snomelt[t, c] \geq 0)$
  - physical_bounds_check.check_snow_melt_heat_flux_not_negative
- $\forall t \forall c(lakestate_vars\%lakeresist_col[t, c] \geq 0)$
  - physical_bounds_check.check_lake_water_transport_resistance_not_negative
- $\forall t \forall c(lakestate_vars\%savedtke1_col[t, c] \geq 0)$
  - physical_bounds_check.check_saved_eddy_conductivity_not_negative
- $\forall t \forall c(col_es\%hc_soi[t, c] \geq 0)$
  - physical_bounds_check.check_soil_heat_content_not_negative
- $\forall t \forall c(col_es\%hc_soisno[t, c] \geq 0)$
  - physical_bounds_check.check_combined_heat_content_not_negative

Range Checks Overview

- Check that a certain model output is in [0, 1]
- 2 out of 55 checks are of this type.

```python
def check_surface_absorption_fraction_is_fraction(
    test_lakestate_vars_betaprime_col: npt.NDArray
) -> None:

    assert_is_frac(test_lakestate_vars_betaprime_col, "lakestate_vars%betaprime_col")
```

Example Check

```python
def assert_is_frac(variable: npt.NDArray, name: str) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(variable):
        return CheckStatus.SKIPPED
    variable = NonFiniteValuesHandler.mask_non_finite_values(variable)
    assert np.all((variable >= 0.0) & (variable <= 1.0)), (
        f"Non-fractional values found in {name}"
    )
```

All Range Checks

All Physical Bounds Checks

- $\forall t \forall j \forall c(\text{lakestate_vars\%lake_icefrac_col}[t, j, c] \in [0, 1])$
  - physical_bounds_check.check_lake_layer_ice_fraction_is_fraction
- $\forall t \forall c(\text{lakestate_vars\%betaprime_col}[t, c] \in [0, 1])$
  - physical_bounds_check.check_surface_absorption_fraction_is_fraction

Finiteness Checks Overview

- Check that a certain model output is finite
- 22 out of 55 checks are of this type.

Example Check

```python
def check_lake_temperature_finite(test_col_es_t_lake: npt.NDArray) -> None:
    assert_variable_is_finite(test_col_es_t_lake, "col_es%t_lake")

def assert_variable_is_finite(variable: npt.NDArray, name: str) -> None:
    assert np.all(np.isfinite(variable)), f"Infinite or NaN values found in {name}"
```

All Finiteness Checks

All Finite Values Checks

- ∀t∀j∀c(col_es%t_lake[t, j, c] != NaN ∩ col_es%t_lake[t, j, c] != inf)
  - finite_values_check.check_lake_temperature_finite
- ∀t∀j∀c(col_es%t_soisno[t, j, c] != NaN ∩ col_es%t_soisno[t, j, c] != inf)
  - finite_values_check.check_soil_snow_temperature_finite
- ∀t∀p(veg_ef%eflx_gnet[t, p] != NaN ∩ veg_ef%eflx_gnet[t, p] != inf)
  - finite_values_check.check_ground_net_heat_flux_finite
- ∀t∀p(veg_ef%eflx_sh_grnd[t, p] != NaN ∩ veg_ef%eflx_sh_grnd[t, p] != inf)
  - finite_values_check.check_ground_sensible_heat_flux_finite
- ∀t∀p(veg_ef%eflx_sh_tot[t, p] != NaN ∩ veg_ef%eflx_sh_tot[t, p] != inf)
  - finite_values_check.check_total_sensible_heat_flux_finite

All Finiteness Checks (Continued)

- ∀t∀p(veg_ef%eflx_soil_grnd[t, p] != NaN ∩ veg_ef%eflx_soil_grnd[t, p] != inf)
  - finite_values_check.check_ground_heat_flux_finite

All Finite Values Checks

- ∀t∀c(col_ef%eflx_snomelt[t, c] != NaN ∩ col_ef%eflx_snomelt[t, c] != inf)
  - finite_values_check.check_snow_melt_heat_flux_finite
- ∀t∀j∀c(col_wf%qflx_snofrz_lyr[t, j, c] != NaN ∩ col_wf%qflx_snofrz_lyr[t, j, c] != inf)
  - finite_values_check.check_snow_freeze_rate_finite
- ∀t∀c(col_wf%qflx_snomelt[t, c] != NaN ∩ col_wf%qflx_snomelt[t, c] != inf)
  - finite_values_check.check_snow_melt_water_flux_finite
- ∀t∀c(col_wf%qflx_snow_melt[t, c] != NaN ∩ col_wf%qflx_snow_melt[t, c] != inf
  - finite_values_check.check_new_snow_melt_rate_finite
- ∀t∀c(col_ws%h2osno[t, c] != NaN ∩ col_ws%h2osno[t, c] != inf)
  - finite_values_check.check_snow_water_equivalent_finite

All Finiteness Checks (Continued)

All Finite Values Checks

- ∀t∀c(col_ws%snow_depth[t, c] != NaN ∩ col_ws%snow_depth[t, c] != inf)
  - finite_values_check.check_snow_depth_finite
- ∀t∀c(lakestate_vars%lakeresist_col[t, c] != NaN ∩ lakestate_vars%lakeresist_col[t, c] != inf)
  - finite_values_check.check_lake_transport_resistance_finite
- ∀t∀c(lakestate_vars%savedtke1_col[t, c] != NaN ∩ lakestate_vars%savedtke1_col[t, c] != inf)
  - finite_values_check.check_saved_eddy_conductivity_finite
- ∀t∀c(lakestate_vars%grnd_ch4_cond_col[t, c] != NaN ∩ lakestate_vars%grnd_ch4_cond_col[t, c] != inf)
  - finite_values_check.check_ground_methane_conductance_finite
  - Precondition: use_lch4 = 1
- ∀t∀c(col_ef%errsoi[t, c] != NaN ∩ col_ef%errsoi[t, c] != inf)
  - finite_values_check.check_energy_conservation_residual_finite

All Finiteness Checks (Continued)

All Finite Values Checks

- ∀t∀j∀c(col_ef%imelt[t, j, c] != NaN ∩ col_ef%imelt[t, j, c] != inf)
  - finite_values_check.check_snow_layer_flag_finite
- ∀t∀c(col_es%hc_soi[t, c] != NaN ∩ col_es%hc_soi[t, c] != inf)
  - finite_values_check.check_soil_heat_content_finite
- ∀t∀c(col_es%hc_soisno[t, c] != NaN ∩ col_es%hc_soisno[t, c] != inf)
  - finite_values_check.check_combined_heat_content_finite
- ∀t∀c(lakestate_vars%betaprime_col[t, c] != NaN ∩ lakestate_vars%betaprime_col[t, c] != inf)
  - finite_values_check.check_surface_absorption_finite
- ∀t∀j∀c(lakestate_vars%lake_icefrac_col[t, j, c] != NaN ∩ lakestate_vars%lake_icefrac_col[t, j, c] != inf)
  - finite_values_check.check_ice_mass_fraction_finite
- ∀t∀c(lakestate_vars%lake_icethick_col[t, c] != NaN ∩ lakestate_vars%lake_icethick_col[t, c] != inf)
  - finite_values_check.check_ice_thickness_finite

Enumerated Value Check Overview

- Check that a certain model output consists of valid enumerated values
- 1 out of 55 checks are of this type.

Example Check

```python
def check_imelt_uses_valid_enum_values(
    test_col_ef_imelt: npt.NDArray,
) -> None:
    # Values of 2147483647 are uninitialized, and should be masked.
    masked_imelt = np.ma.masked_where(
        test_col_ef_imelt == 2147483647, test_col_ef_imelt
    )
    if NonFiniteValuesHandler.is_all_not_finite(masked_imelt):
        return CheckStatus.SKIPPED
    masked_imelt = NonFiniteValuesHandler.mask_non_finite_values(masked_imelt)

    assert np.all(masked_imelt >= 0) and np.all(masked_imelt <= 2)
```

All Enumerated Value Checks

- $\forall t \forall j \forall c(col_ef\%imelt[t, j, c] \in \{0, 1, 2\})$ Aggregation and Consistency Check
  - aggregation_and_consistency_check.check_imelt_uses_valid_enum_values

Physical Laws and Constraints Checks Overview

- Check that a certain model output obeys physical laws and constraints
- 19 out of 55 checks are of this type.

Example Check

```python
def check_temp_around_freezing_where_lake_is_almost_frozen(
    test_col_es_t_lake: npt.NDArray,
    test_lakestate_vars_lake_icefrac_col: npt.NDArray,
) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(test_col_es_t_lake,
                                                test_lakestate_vars_lake_icefrac_col):
        return CheckStatus.SKIPPED
    test_col_es_t_lake, test_lakestate_vars_lake_icefrac_col = (
        NonFiniteValuesHandler.mask_non_finite_values(test_col_es_t_lake,
                                                      test_lakestate_vars_lake_icefrac_col))

    almost_frozen = (np.isclose(test_lakestate_vars_lake_icefrac_col, 1.0)
                     & (test_lakestate_vars_lake_icefrac_col != 1.0))

    if not np.any(almost_frozen):
        return CheckStatus.SKIPPED

    diff_to_freezing_temp = np.abs(TFRZ - test_col_es_t_lake)
    close_to_freezing_temp = diff_to_freezing_temp <= 1E-3

    assert np.all(~almost_frozen | close_to_freezing_temp), (
        "Columns are almost frozen but not close to the freezing temperature"
    )
```

All Physical Laws and Constraints Checks

- $\forall t \forall c((\sum_j \text{lakestate_vars\%lake_icefrac_col}[t, j, c] \times \text{col_pp\%dz_lake}[t, j, c] \times \text{denh2o} \div \text{denice}) \approx \text{lakestate_vars\%lake_icethick_col}[t, c])$ Aggregation and Consistency Check
  - aggregation_and_consistency_check.check_icethick_col_is_sum
  - Tolerance: 1E-9
- Equation
  - physical_bounds_check.check_combined_heat_content_not_less_than_soil_heat_content
  - Tolerance:

All Physical Laws and Constraints Checks (Continued)

- $\forall t \forall c(\sum_j \text{col_wf\%qflx_snofrz_lyr}[t, c] \approx \text{col_wf\%qflx_snofrz}[t, c])$
  - aggregation_and_consistency_check.check_snofrz_lyr_sums_to_snofrz_col
  - Tolerance: 1E-10

Aggregation and Consistency Check

- $\forall t \forall c((\text{precondition}) \rightarrow \text{col_ef\%eflx_snomelt}[t, c] \approx \text{col_wf\%qflx_snomelt}[t, c] \times \text{hfus})$
  - key_physical_laws_check.check_energy_flux_consistent_with_latent_heat
  - Tolerance: 1E-6

Snow Melting Check

- $\forall t \forall c(\text{lakestate_vars\%lake_icefrac_col}[t, 0, c] = 0 \rightarrow (1/(\text{lakestate_vars\%lakeresist_col}[t, c] + \text{lakestate_vars\%lake_raw_col}[t, c])) \approx \text{ch4_vars\%grnd_ch4_cond_col}[t, c])$
  - ch4_conductance_check.check_methane_conductance_allowed_without_ice
  - Precondition: use_lch4 = 1
  - Tolerance: 1E-6

CH4 Conductance Behavior Check

All Physical Laws and Constraints Checks (Continued)

- $\forall t \forall c((\text{precondition}) \rightarrow (\Delta(\text{col_ws\%snow_depth}[t, c])/\Delta t \leq 0))$
  - key_physical_laws_check.check_snow_depth_not_increasing_while_melting
  - Tolerance: 1E-6

All Snow Melting Checks

All Physical Laws and Constraints Checks (Continued)

All Radiation Absorption Checks

- $\forall t \forall c(\text{col_pp\%snl}[t, c] = 0 \rightarrow \text{lakestate_vars\%betaprime_col}[t, c] \approx \sum_p(\text{solarabs_vars\%sabg_lyr_patch}[t, 0, p] \div \text{solarabs_vars\%sabg_patch}[t, p]))$, where $\text{veg_pp\%column}[t, p] = c)$
  - key_physical_laws_check.check_betaprime_matches_nir_blend
  - Tolerance: 1E-6
- Equation
  - physics_check.check_fully_frozen_layers_not_above_freezing
  - Tolerance:

All Physical Laws and Constraints Checks (Continued)

- $\forall t \forall c(\text{lakestate_vars\%lake_icefrac_col}[t, 0, c] > 0.1 \rightarrow \text{ch4_vars\%grnd_ch4_cond_col}[t, c] \approx 0)$
  - ch4_conductance_check.check_methane_conductance_gated_by_ice
  - Tolerance: 1E-5
  - Precondition: $\text{use_lch4} = 1$

CH4 Conductance Behavior Check

- Precondition $\rightarrow$ $\forall t \forall c(\text{col_ef\%errsoi}[t, c] \approx 0)$
  - key_physical_laws_check.check_errsoi_threshold
  - Tolerance: 1E-6

Energy Conservation Check

- Precondition $\rightarrow$ $\forall t \forall c(\text{col_pp\%snl}[t, c] < 0 \cap \text{col_ws\%h2osno}[t, c] > 0 \rightarrow \text{col_wf\%qflx_snofrz_lyr}[t, 0, c] > 0)$
  - key_physical_laws_check.check_surface_snow_freezing_where_snow_present
- Precondition -> $\forall t \forall c(\text{col_pp\%snl}[t, c] < 0 \cap \text{col\%ws_h2osno}[t, c] > 0 \rightarrow \text{col_ef\%imelt}[t, 0, c] = 2)$
  - key_physical_laws_check.check_snow_labeled_freezing_where_snow_present

Snow Freezing Check

Snow Freezing Check

All Physical Laws and Constraints Checks (Continued)

- Precondition -> $\forall t \forall c(col_pp\%snl[t, c] < 0 \cap col_ws\%h2osno[t, c] > 0 \rightarrow col_wf\%qflx_snomelt[t,c] = 0)$
  - key_physical_laws_check.check_snow_not_melting_where_snow_present

Snow Freezing Check

- Precondition -> $\forall t \forall c(\Delta(col_ws\%h2osno[t, c])/\Delta t \geq 0)$
  - key_physical_laws_check.check_snow_water_not_decreasing
- Precondition $\rightarrow$ $\forall t \forall c(\Delta(col_ws\%snow_depth[t, c])/\Delta t \geq 0)$
  - key_physical_laws_check.check_snow_depth_not_decreasing

Snow Freezing Check

- Precondition $\rightarrow$ $\forall t \forall c(col_pp\%snl[t, 0, c] = 0 \cap col_ws\%h2osno[t, c] > 0 \cap col_es\%t_lake[t, 0, c] > tfrz \rightarrow col_wf\%qflx_snomelt[t, c] > 0)$
  - key_physical_laws_check.check_snow_melting_where_snow_water_present

Snow Freezing Check

Snow Melting Check

All Physical Laws and Constraints Checks (Continued)

- Precondition $\rightarrow \forall t \forall c(col_pp\%snl[t, c] = 0 \cap col_ws\%h2osno[t, c] > 0 \cap col_es\%t_lake[t, 0, c] > tfrz \rightarrow col_wf\%qflx_snow_melt[t, c] > 0)$
  - key_physical_laws_check.check_snow_melted_where_snow_water_present

Snow Melting Check

- $\forall t \forall j \forall c(lakestate_vars\%lake_icefrac_col[t, j, c] > 0.999 \cap lakestate_vars\%lake_icefrac_col[t, j, c] \neq 1 \rightarrow col_es\%t_lake[t, j, c] - tfrz \approx 0)$
  - physics_check.check_temp_around_freezing_where_lake_is_almost_frozen
  - Tolerance: 1E-3

CH4 Conductance Behavior Check

- $\forall t \forall c(lakestate_vars\%savedtke1_col[t, c] > 0 \rightarrow lakestate_vars\%lake_icefrac_col[t, 0, c] \approx 0 \cap col_pp\%snl[t, c] = 0)$
  - physics_check.check_surface_unfrozen_when_tke_present
  - Tolerance: 1E-6

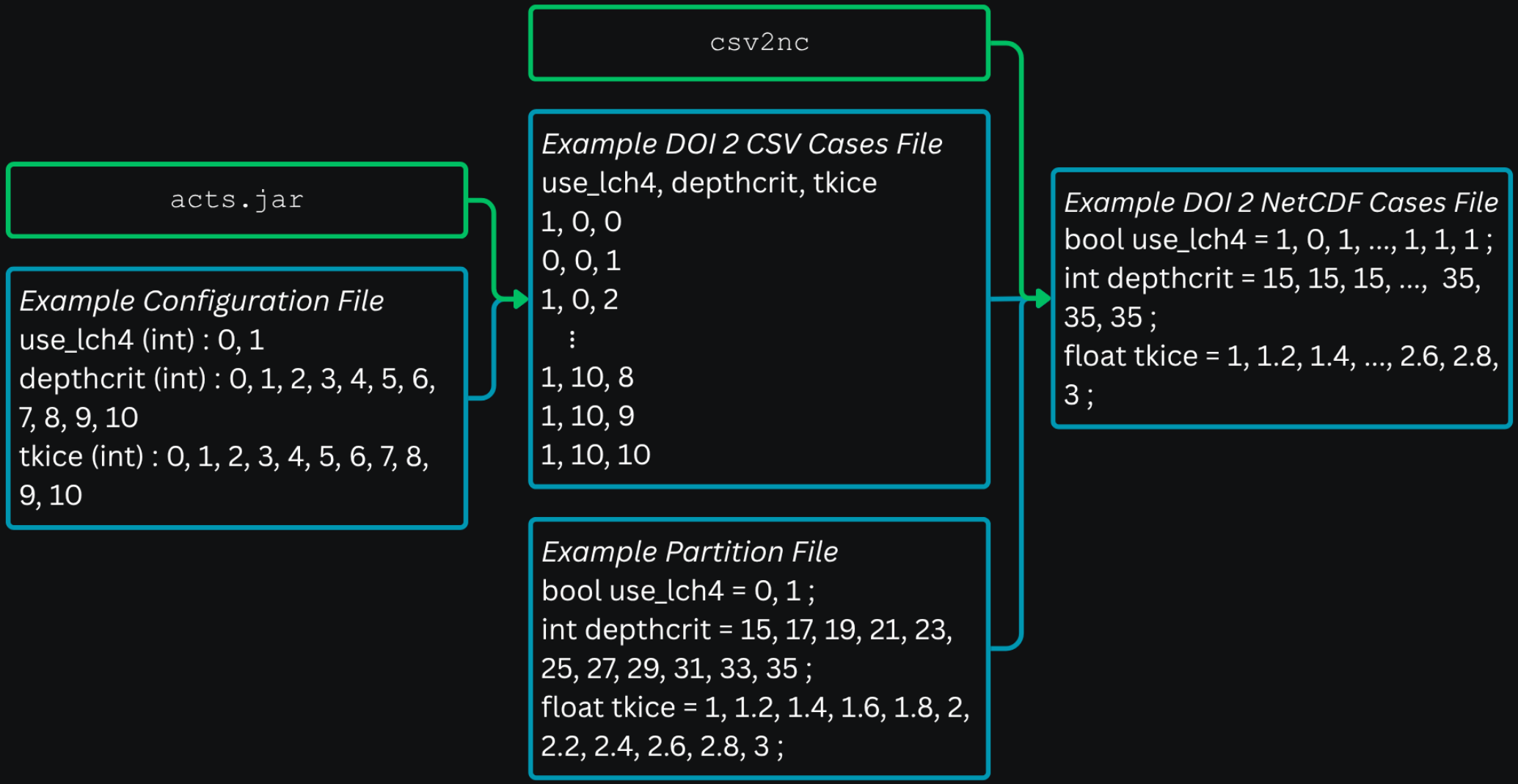
Physics Sanity Check

Physics Sanity Check

Workflow for Sample Generation

- The configuration file provides the indices for the number of values each model constant can take.
- ACTS uses these indices to generate covering arrays from 2-way to 5-way and outputs a CSV file for each.
- The partition file provides the values that correspond to the indices in the csv files.
- The Python script (`csv2nc`) uses the partition file's values to create NetCDF files which are run on the model using the NetCDF cases sampling plugin.




csv2nc
acts.jar
Example Configuration File
use_lch4 (int) : 0, 1
depthcrit (int) : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
tkice (int) : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Example DOI 2 CSV Cases File
use_lch4, depthcrit, tkice
1, 0, 0
0, 0, 1
1, 0, 2
⋮
1, 10, 8
1, 10, 9
1, 10, 10
Example Partition File
bool use_lch4 = 0, 1 ;
int depthcrit = 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35 ;
float tkice = 1, 1.2, 1.4, 1.6, 1.8, 2, 2.2, 2.4, 2.6, 2.8, 3 ;
Example DOI 2 NetCDF Cases File
bool use_lch4 = 1, 0, 1, ..., 1, 1, 1 ;
int depthcrit = 15, 15, 15, ..., 35, 35, 35 ;
float tkice = 1, 1.2, 1.4, ..., 2.6, 2.8, 3 ;

Test Suite Sizes and Timings

For ten samples, the mean time it takes to run `elmtest` (for 35 time steps) is 0.934 seconds, with a standard deviation of 0.009. This time is used to estimate how long testing each interaction level would take, assuming tests run sequentially.

*Large test cases haven't been executed yet.

DOI	Case Count	Estimated Execution Time	Actual Execution Time
1	11	0 days, 00:00:10	0 days, 00:00:16
2	232	0 days, 00:03:37	0 days, 00:05:02
3	4,253	0 days, 01:06:12	0 days, 00:52:30
4	64,149	0 days, 16:38:35	- days, --:--:--*
5	869,835	9 days, 09:40:26	- days, --:--:--*

ACTS Case Generation Timings

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
1	11	0.226	0.003
2	232	0.345	0.006
3	4,253	3.305	0.029
4	64,149	373.978	4.900
5	869,835	4311.719	54.021

- This represents the amount of time it took to generate the csv test cases with all scalar model constants using ACTS.
- The mean and standard deviation were calculated using 7 runs of ACTS for each DOI level.

`csv2nc` Script Timings

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
1	11	0.115	0.001
2	232	0.307	0.004
3	4,253	3.873	0.082
4	64,149	59.630	3.912
5	869,835	763.368	7.505

- This represents the amount of time it took to convert the csv test cases with all scalar model constants to NetCDF using the `csv2nc` script.
- The mean and standard deviation were calculated using 7 runs of `csv2nc` for each DOI level.

DOI 1 Results

- Every check, except 2, either passed or skipped very every test case.
- check_surface_unfrozen_when_tke_present and check_temp_around_freezing_where_lake_is_almost_frozen failed on the test case where cnfac == 1

0 out of 11 runs exited with an error.

Check	Passed	Skipped	Failed	Errored
check_betaprime_matches_nir_blend	11	0	0	0
check_combined_heat_content_finite	11	0	0	0
check_combined_heat_content_not_less_than_soil_heat_content	11	0	0	0
check_combined_heat_content_not_negative	11	0	0	0
check_energy_conservation_residual_finite	11	0	0	0
check_energy_flux_consistent_with_latent_heat	11	0	0	0
check_errsoi_threshold	11	0	0	0
check_fully_frozen_layers_not_above_freezing	0	11	0	0
check_ground_heat_flux_finite	11	0	0	0
check_ground_methane_conductance_finite	4	7	0	0
check_ground_net_heat_flux_finite	11	0	0	0
check_ground_sensible_heat_flux_finite	11	0	0	0
check_ice_mass_fraction_finite	11	0	0	0
check_ice_thickness_finite	11	0	0	0
check_icethick_col_is_sum	11	0	0	0
check_imelt_uses_valid_enum_values	11	0	0	0
check_lake_layer_ice_fraction_is_fraction	11	0	0	0
check_lake_temperature_finite	11	0	0	0
check_lake_transport_resistance_finite	11	0	0	0
check_lake_water_transport_resistance_not_negative	11	0	0	0
check_methane_conductance_allowed_without_ice	0	11	0	0
check_methane_conductance_gated_by_ice	4	7	0	0
check_methane_conductance_not_negative	4	7	0	0
check_net_snow_melt_not_negative	11	0	0	0
check_new_snow_melt_rate_finite	11	0	0	0
check_saved_eddy_conductivity_finite	11	0	0	0
check_saved_eddy_conductivity_not_negative	11	0	0	0
check_snofrz_lyr_sums_to_snofrz_col	11	0	0	0
check_snow_depth_finite	11	0	0	0
check_snow_depth_not_decreasing	11	0	0	0
check_snow_depth_not_increasing_while_melting	11	0	0	0
check_snow_depth_not_negative	11	0	0	0
check_snow_freeze_rate_finite	11	0	0	0
check_snow_freeze_rate_not_negative	11	0	0	0
check_snow_labeled_freezing_where_snow_present	11	0	0	0
check_snow_layer_flag_finite	11	0	0	0
check_snow_melt_flux_not_negative	11	0	0	0
check_snow_melt_heat_flux_finite	11	0	0	0
check_snow_melt_heat_flux_not_negative	11	0	0	0
check_snow_melt_water_flux_finite	11	0	0	0
check_snow_melted_where_snow_water_present	11	0	0	0
check_snow_melting_where_snow_water_present	11	0	0	0
check_snow_not_melting_where_snow_present	11	0	0	0
check_snow_water_equivalent_finite	11	0	0	0
check_snow_water_not_decreasing	11	0	0	0
check_soil_heat_content_finite	11	0	0	0
check_soil_heat_content_not_negative	11	0	0	0
check_soil_snow_temperature_finite	11	0	0	0
check_surface_absorption_finite	11	0	0	0
check_surface_absorption_fraction_is_fraction	11	0	0	0
check_surface_snow_freezing_where_snow_present	11	0	0	0
check_surface_unfrozen_when_tke_present	10	0	1	0
check_temp_around_freezing_where_lake_is_almost_frozen	0	10	1	0
check_total_sensible_heat_flux_finite	11	0	0	0
check_water_snow_equivalent_not_negative	11	0	0	0

DOI 2 Results

- Every check, except 2, either passed or skipped very every test case.
- check_surface_unfrozen_when_tke_present and check_temp_around_freezing_where_lake_is_almost_frozen failed on all test cases where cnfac == 1

0 out of 232 runs exited with an error.

Check	Passed	Skipped	Failed	Errored
check_betaprime_matches_nir_blend	232	0	0	0
check_combined_heat_content_finite	232	0	0	0
check_combined_heat_content_not_less_than_soil_heat_content	232	0	0	0
check_combined_heat_content_not_negative	232	0	0	0
check_energy_conservation_residual_finite	232	0	0	0
check_energy_flux_consistent_with_latent_heat	232	0	0	0
check_errsoi_threshold	232	0	0	0
check_fully_frozen_layers_not_above_freezing	0	232	0	0
check_ground_heat_flux_finite	232	0	0	0
check_ground_methane_conductance_finite	116	116	0	0
check_ground_net_heat_flux_finite	232	0	0	0
check_ground_sensible_heat_flux_finite	232	0	0	0
check_ice_mass_fraction_finite	232	0	0	0
check_ice_thickness_finite	232	0	0	0
check_icethick_col_is_sum	232	0	0	0
check_imelt_uses_valid_enum_values	232	0	0	0
check_lake_layer_ice_fraction_is_fraction	232	0	0	0
check_lake_temperature_finite	232	0	0	0
check_lake_transport_resistance_finite	232	0	0	0
check_lake_water_transport_resistance_not_negative	232	0	0	0
check_methane_conductance_allowed_without_ice	0	232	0	0
check_methane_conductance_gated_by_ice	116	116	0	0
check_methane_conductance_not_negative	116	116	0	0
check_net_snow_melt_not_negative	232	0	0	0
check_new_snow_melt_rate_finite	232	0	0	0
check_saved_eddy_conductivity_finite	232	0	0	0
check_saved_eddy_conductivity_not_negative	232	0	0	0
check_snofrz_lyr_sums_to_snofrz_col	232	0	0	0
check_snow_depth_finite	232	0	0	0
check_snow_depth_not_decreasing	232	0	0	0
check_snow_depth_not_increasing_while_melting	232	0	0	0
check_snow_depth_not_negative	232	0	0	0
check_snow_freeze_rate_finite	232	0	0	0
check_snow_freeze_rate_not_negative	232	0	0	0
check_snow_labeled_freezing_where_snow_present	232	0	0	0
check_snow_layer_flag_finite	232	0	0	0
check_snow_melt_flux_not_negative	232	0	0	0
check_snow_melt_heat_flux_finite	232	0	0	0
check_snow_melt_heat_flux_not_negative	232	0	0	0
check_snow_melt_water_flux_finite	232	0	0	0
check_snow_melted_where_snow_water_present	232	0	0	0
check_snow_melting_where_snow_water_present	232	0	0	0
check_snow_not_melting_where_snow_present	232	0	0	0
check_snow_water_equivalent_finite	232	0	0	0
check_snow_water_not_decreasing	232	0	0	0
check_soil_heat_content_finite	232	0	0	0
check_soil_heat_content_not_negative	232	0	0	0
check_soil_snow_temperature_finite	232	0	0	0
check_surface_absorption_finite	232	0	0	0
check_surface_absorption_fraction_is_fraction	232	0	0	0
check_surface_snow_freezing_where_snow_present	232	0	0	0
check_surface_unfrozen_when_tke_present	211	0	21	0
check_temp_around_freezing_where_lake_is_almost_frozen	0	211	21	0
check_total_sensible_heat_flux_finite	232	0	0	0
check_water_snow_equivalent_not_negative	232	0	0	0

DOI 3 Results

- Every check, except 2, either passed or skipped very every test case.
- check_surface_unfrozen_when_tke_present and check_temp_around_freezing_where_lake_is_almost_frozen failed on almost all test cases where cnfac == 1
- These two checks passed only twice when cnfac == 1. This was on the two test cases where cnfac == 1, pudz == 0, and lakepuddling == 1 == true

0 out of 4253 runs exited with an error.

Check	Passed	Skipped	Failed	Errored
check_betaprime_matches_nir_blend	4253	0	0	0
check_combined_heat_content_finite	4253	0	0	0
check_combined_heat_content_not_less_than_soil_heat_content	4253	0	0	0
check_combined_heat_content_not_negative	4253	0	0	0
check_energy_conservation_residual_finite	4253	0	0	0
check_energy_flux_consistent_with_latent_heat	4253	0	0	0
check_errsoi_threshold	4253	0	0	0
check_fully_frozen_layers_not_above_freezing	0	4253	0	0
check_ground_heat_flux_finite	4253	0	0	0
check_ground_methane_conductance_finite	2125	2128	0	0
check_ground_net_heat_flux_finite	4253	0	0	0
check_ground_sensible_heat_flux_finite	4253	0	0	0
check_ice_mass_fraction_finite	4253	0	0	0
check_ice_thickness_finite	4253	0	0	0
check_icethick_col_is_sum	4253	0	0	0
check_imelt_uses_valid_enum_values	4253	0	0	0
check_lake_layer_ice_fraction_is_fraction	4253	0	0	0
check_lake_temperature_finite	4253	0	0	0
check_lake_transport_resistance_finite	4253	0	0	0
check_lake_water_transport_resistance_not_negative	4253	0	0	0
check_methane_conductance_allowed_without_ice	0	4253	0	0
check_methane_conductance_gated_by_ice	2125	2128	0	0
check_methane_conductance_not_negative	2125	2128	0	0
check_net_snow_melt_not_negative	4253	0	0	0
check_new_snow_melt_rate_finite	4253	0	0	0
check_saved_eddy_conductivity_finite	4253	0	0	0
check_saved_eddy_conductivity_not_negative	4253	0	0	0
check_snofrz_lyr_sums_to_snofrz_col	4253	0	0	0
check_snow_depth_finite	4253	0	0	0
check_snow_depth_not_decreasing	4253	0	0	0
check_snow_depth_not_increasing_while_melting	4253	0	0	0
check_snow_depth_not_negative	4253	0	0	0
check_snow_freeze_rate_finite	4253	0	0	0
check_snow_freeze_rate_not_negative	4253	0	0	0
check_snow_labeled_freezing_where_snow_present	4253	0	0	0
check_snow_layer_flag_finite	4253	0	0	0
check_snow_melt_flux_not_negative	4253	0	0	0
check_snow_melt_heat_flux_finite	4253	0	0	0
check_snow_melt_heat_flux_not_negative	4253	0	0	0
check_snow_melt_water_flux_finite	4253	0	0	0
check_snow_melted_where_snow_water_present	4253	0	0	0
check_snow_melting_where_snow_water_present	4253	0	0	0
check_snow_not_melting_where_snow_present	4253	0	0	0
check_snow_water_equivalent_finite	4253	0	0	0
check_snow_water_not_decreasing	4253	0	0	0
check_soil_heat_content_finite	4253	0	0	0
check_soil_heat_content_not_negative	4253	0	0	0
check_soil_snow_temperature_finite	4253	0	0	0
check_surface_absorption_finite	4253	0	0	0
check_surface_absorption_fraction_is_fraction	4253	0	0	0
check_surface_snow_freezing_where_snow_present	4253	0	0	0
check_surface_unfrozen_when_tke_present	3872	0	381	0
check_temp_around_freezing_where_lake_is_almost_frozen	2	3870	381	0
check_total_sensible_heat_flux_finite	4253	0	0	0
check_water_snow_equivalent_not_negative	4253	0	0	0

Results Explanation

- These checks were run using a 35 time step limit.
- Some checks were skipped because they are only valid under certain preconditions. For example, check_methane_conductance_allowed_without_ice cannot run when use_lch4 == 0, when methane is turned off.
- We cannot ensure some of the preconditions are met because they rely on the starting values of some model inputs, which we cannot control.
- Every check makes an assertion about model output data. When that assertion fails, the check fails. This can point to a problem in the design of the check, SPEL, or `LakeTemperature`. A check outputs an error when it is unable to run correctly, e.g., if there was a syntax mistake. Errors point to bugs in our code that will be fixed prior to final testing.

Findings from Preliminary Testing

- Largely, `LakeTemperature` output data appeared valid and within physical bounds. However, some cases resulted in physically impossible outputs, likely due to the interactions between model constants.
- A potential issue we found is that cnfac == 1 seems to cause invalid output, unless it is also paired with pudz == 0 and lakepuddling == 1 == true

Limitations and Next Steps

- So far, only combinatorial test cases 1-3 have been run. Next, 4 and 5 will be run to expose more potential issues caused by interactions of higher numbers of model constants.
- So far, `elmtest.exe` has only been run with 35 timesteps. To get more in depth results, it will be run with 1000 timesteps.

Part 4: Boundary Testing

Boundary Testing Overview

- Boundary testing allowed us to validate the ranges we were given for each of the model constants and discover whether values outside of the range would cause invalid model output.
- For this testing method, we paired order sampling, to linearly interpolate above and below the top and bottom of the range for a model constant, with fault checks, to ensure the model output data is valid.
- We chose to investigate betavis and cnfac, as betavis is a model constant the domain scientists are very familiar with and our results from combinatorial testing seemed to imply the top of cnfac's range caused invalid model output data. Both constant's ranges are give as [0, 1] so we interpolated across -0.1 – 0.1 and 0.9 – 1.1 for both.

Boundary Testing Results

betavis

Check Failed	Values
check_surface_absorption_fraction_is_fraction	-0.1, 1.1
None	0, 0.1, 0.9, 1.0

cnfac

Check Failed	Values
check_surface_unfrozen_when_tke_present	1.0, 1.1
check_temp_around_freezing_where_lake_is_almost_frozen	1.0, 1.1
None	-0.1, 0, 0.1, 0.9

Boundary Testing Findings

- Our results for betavis validated the range we were given for it and demonstrated that the range [0, 1] for betavis does not produce invalid model output data. As we'd expect, values outside of that range do.
- Our results for cnfac indicate that values where cnfac < 0 do not necessarily produce invalid model output data. Using values where cnfac ≥ 1 does produce invalid model output data. This does not necessarily mean that the range [0, 1] for cnfac is incorrect but might expose that the `LakeTemperature` module produces invalid output data when cnfac == 1